

01 UART Specification

- [SMC UART 16550 — Low-Level SystemC Implementation](#)
 - [Contents](#)
 - [1. Purpose and scope](#)
 - [2. Source layout](#)
 - [3. Module structure](#)
 - [3.1 CCI parameter ordering](#)
 - [3.2 Public methods \(API\)](#)
 - [3.3 Internal methods and SC processes](#)
 - [4. TLM-2.0 interface implementation](#)
 - [4.1 Socket and payload](#)
 - [4.2 Error-response mapping](#)
 - [4.3 transport_dbg](#)
 - [4.4 DMI](#)
 - [5. Register decode](#)
 - [5.1 DLAB-aware decode table](#)
 - [5.2 Read side effects](#)
 - [5.3 Write side effects](#)
 - [6. Internal data structures](#)
 - [6.1 Register file](#)
 - [6.2 FIFO entries](#)
 - [6.3 Derived/runtime state](#)
 - [7. Process and event topology](#)
 - [7.1 RX timeout timer](#)
 - [8. Behavioural algorithms](#)
 - [8.1 Transmit](#)
 - [8.2 Receive](#)
 - [8.3 Interrupt arbitration](#)
 - [8.4 DMA handshake](#)
 - [8.5 Modem control / loopback](#)
 - [9. Reset implementation](#)
 - [10. Loosely-timed timing model and quantum keeper](#)
 - [10.1 Timing philosophy](#)
 - [10.2 Role of the quantum keeper](#)
 - [10.3 Test-bench initiator pattern](#)
 - [11. Error handling implementation](#)
 - [12. Debug and verification hooks](#)
 - [13. Integration with smc_top](#)
 - [14. Modeling decisions and trade-offs](#)
 - [15. Known limitations and future work](#)

SMC UART 16550 — Low-Level SystemC Implementation

Document: 02_UART_LowLevel_Design.md **Module:** smc::uart (peripherals/uart/) **SystemC:** 3.0.2 Accellera (compatible with $\geq 2.3.4$), TLM-2.0 **RTL reference:** hw/comp/uart_16550/rtl/ (uart_16550.sv, uart_core.sv, uart_tx.sv, uart_rx.sv) **Status:** Design phase — defines the planned internal implementation; will match include/uart.h and src/uart.cpp **Companion docs:** - 01_UART_Specification.md — externally-observable behaviour - 03_UART_Test_Plan.md — verification strategy and test list

Contents

1. [Purpose and scope](#)
 2. [Source layout](#)
 3. [Module structure](#)
 4. [TLM-2.0 interface implementation](#)
 5. [Register decode](#)
 6. [Internal data structures](#)
 7. [Process and event topology](#)
 8. [Behavioural algorithms](#)
 9. [Reset implementation](#)
 10. [Loosely-timed timing model and quantum keeper](#)
 11. [Error handling implementation](#)
 12. [Debug and verification hooks](#)
 13. [Integration with smc_top](#)
 14. [Modeling decisions and trade-offs](#)
 15. [Known limitations and future work](#)
-

1. Purpose and scope

[01_UART_Specification.md](#) defines **what** the UART looks like from the outside — sockets, signals, register layout, protocol semantics. This document defines **how** the SystemC/TLM-2.0 loosely-timed functional model is implemented internally:

- the data structures it owns,
- the SystemC process topology that drives them,
- the algorithms for transmit, receive, interrupt arbitration, and DMA handshaking,
- the loosely-timed timing strategy and the role of the quantum keeper,
- the verification hooks exposed to the test bench, and
- the integration recipe required to drop the model into `smc_top`.

It is the implementation contract that any future maintainer should be able to read in isolation.

2. Source layout

```
peripherals/uart/
├─ CMakeLists.txt
├─ README.md
├─ run_tests.sh           Build + run convenience script
├─ deps.env               Dependency versions (SystemC, CCI)
├─ doc/
│   ├─ 01_UART_Specification.md
│   ├─ 02_UART_LowLevel_Design.md (this file)
│   └─ 03_UART_Test_Plan.md
│   └─ build_docs.sh
├─ include/
│   ├─ uart.h             SC_MODULE(uart) declaration
│   └─ smc_tlm_extensions.h Shared GP extension (smc_axi_extension)
├─ src/
│   └─ uart.cpp           Implementation
├─ test/
│   ├─ CMakeLists.txt
│   └─ uart_tb.cpp        Self-checking test bench
└─ build/                 (out-of-source CMake build tree)
```

Estimated payload: ~1100 lines of C++20 incl. tests, ~620 lines excl. tests.

3. Module structure

```

#include <cci_configuration> // OSCI CCI - cci_param, cci_broker_handle
#include <tlm_utils/simple_target_socket.h>
#include <tlm_utils/tlm_quantumkeeper.h>

namespace smc {

struct uart_cfg {
    unsigned tx_fifo_depth = 32; // CCI default
    unsigned rx_fifo_depth = 32; // CCI default

    static constexpr uint64_t WINDOW_SIZE = 0x100; // 256-byte window
    static constexpr unsigned REG_WIDTH = 4; // register stride (bytes)
};

class uart : public sc_core::sc_module {
protected:
    // CCI params - declared BEFORE any sc_vector / sized members so they
    // are initialised first.
    cci::cci_param<unsigned, cci::CCI_IMMUTABLE_PARAM> tx_fifo_depth_p_;
    cci::cci_param<unsigned, cci::CCI_IMMUTABLE_PARAM> rx_fifo_depth_p_;
    cci::cci_param<double> access_delay_ns_p_;

public:
    SC_HAS_PROCESS(uart);

    tlm_utils::simple_target_socket<uart> reg_socket;

    sc_core::sc_in<bool> rst_n_i;
    sc_core::sc_out<bool> tx_o;
    sc_core::sc_in<bool> rx_i;

    // Modem interface (active-low)
    sc_core::sc_in<bool> cts_ni, dsr_ni, ri_ni, dcd_ni;
    sc_core::sc_out<bool> rts_no, dtr_no, out1_no, out2_no;

    // DMA / status / interrupt
    sc_core::sc_out<bool> rxrdy_o;
    sc_core::sc_out<bool> txrdy_o;
    sc_core::sc_out<bool> err_o;
    sc_core::sc_out<bool> irq_o;

    explicit uart(sc_core::sc_module_name name, uart_cfg cfg = uart_cfg{});

    // Test-bench back door (no socket, no bus side effects)
    void inject_rx_char(uint8_t ch, bool parity_err = false,
                       bool framing_err = false, bool break_err = false);
    bool dbg_tx_pop(uint8_t& ch); // pop a transmitted character
    unsigned dbg_tx_count() const; // chars waiting in TX FIFO
    unsigned dbg_rx_count() const; // chars waiting in RX FIFO
    uint32_t dbg_reg(uint64_t off) const; // peek a register (no side effects)
    void dump_state(std::ostream& = std::cout) const;

private:
    void b_transport(tlm::tlm_generic_payload&, sc_core::sc_time&);

```

```

    unsigned int transport_dbg(tlm::tlm_generic_payload&);

    void reset_proc();           // SC_METHOD on rst_n_i
    void modem_method();         // SC_METHOD on cts_ni/dsr_ni/ri_ni/dcd_ni
    void recompute_method();     // SC_METHOD on recompute_event_
    void rx_timeout_method();    // SC_METHOD on rx_timeout_event_

    bool reg_read (uint64_t off, uint32_t& data);
    bool reg_write(uint64_t off, uint32_t data);

    void update_outputs();       // single-driver point for irq/dma/err/modem
    void schedule_recompute();
    uint8_t interrupt_id() const;
    bool    any_interrupt() const;
};

} // namespace smc

```

3.1 CCI parameter ordering

The three `cci_param` members are declared **before** the FIFO storage so that their resolved values are available when the constructor sizes the internal FIFO containers. The constructor logs each resolved value with an `SC_REPORT_INFO` tag of `[preset]` or `[default]`.

3.2 Public methods (API)

Method	Purpose
<code>inject_rx_char()</code>	Back door for the test bench to deliver a received character with optional error flags, bypassing the (unmodelled) serial line.
<code>dbg_tx_pop()</code>	Back door to pop the next transmitted character (the model's "serial output").
<code>dbg_tx_count()</code> / <code>dbg_rx_count()</code>	FIFO occupancy queries.
<code>dbg_reg()</code>	Side-effect-free register peek.
<code>dump_state()</code>	Human-readable dump for debugging.

3.3 Internal methods and SC processes

Member	Kind	Sensitivity	Role
<code>b_transport</code>	TLM callback	—	Register read/write entry point
<code>transport_dbg</code>	TLM callback	—	Side-effect-free debug access
<code>reset_proc</code>	SC_METHOD	<code>rst_n_i</code>	Clears all state on active-low reset
<code>modem_method</code>	SC_METHOD	<code>cts_ni</code> , <code>dsr_ni</code> , <code>ri_ni</code> , <code>dcd_ni</code>	Detects modem-input changes, updates MSR delta bits
<code>recompute_method</code>	SC_METHOD	<code>recompute_event_</code>	Recomputes interrupt/DMA/status outputs
<code>rx_timeout_method</code>	SC_METHOD	<code>rx_timeout_event_</code>	Fires the RX timeout interrupt
<code>update_outputs</code>	helper	—	Single writer of all <code>sc_out</code> ports

4. TLM-2.0 interface implementation

4.1 Socket and payload

The model exports one `tlm_utils::simple_target_socket<uart>` named `reg_socket`. The initiator (CPU/fabric) issues `b_transport` calls carrying a `tlm_generic_payload`.

`b_transport` performs, in order:

```

void uart::b_transport(tlm::tlm_generic_payload& gp, sc_core::sc_time& delay) {
    const uint64_t addr = gp.get_address();
    const unsigned len = gp.get_data_length();
    uint8_t* ptr = gp.get_data_ptr();

    // 1. Command check
    const auto cmd = gp.get_command();
    if (cmd != tlm::TLM_READ_COMMAND && cmd != tlm::TLM_WRITE_COMMAND) {
        gp.set_response_status(tlm::TLM_COMMAND_ERROR_RESPONSE);
        return;
    }
    // 2. Width check (32-bit register access only)
    if (len != 4) {
        gp.set_response_status(tlm::TLM_BURST_ERROR_RESPONSE);
        return;
    }
    // 3. Window + alignment check
    if (addr >= uart_cfg::WINDOW_SIZE || (addr & 0x3) != 0) {
        gp.set_response_status(tlm::TLM_ADDRESS_ERROR_RESPONSE);
        return;
    }
    // 4. Dispatch
    bool ok;
    if (cmd == tlm::TLM_READ_COMMAND) {
        uint32_t data = 0;
        ok = reg_read(addr, data);
        std::memcpy(ptr, &data, 4);
    } else {
        uint32_t data;
        std::memcpy(&data, ptr, 4);
        ok = reg_write(addr, data);
    }
    // 5. Annotate loosely-timed delay
    delay += sc_core::sc_time(access_delay_ns_p_.get_value(), sc_core::SC_NS);

    gp.set_response_status(ok ? tlm::TLM_OK_RESPONSE
                           : tlm::TLM_ADDRESS_ERROR_RESPONSE);
}

```

4.2 Error-response mapping

Condition	Response
Command not READ/WRITE	TLM_COMMAND_ERROR_RESPONSE
data_length != 4	TLM_BURST_ERROR_RESPONSE
addr >= WINDOW_SIZE or unaligned	TLM_ADDRESS_ERROR_RESPONSE
Decode miss inside window	TLM_ADDRESS_ERROR_RESPONSE
Success	TLM_OK_RESPONSE

4.3 transport_dbg

transport_dbg calls a side-effect-free path: register reads do **not** pop the RX FIFO, do **not** clear IIR/MSR sticky bits, and do **not** advance any state. It returns 4 on a valid access and 0 otherwise. This is the access

path used by debuggers and by the test bench's `dbg_reg()`.

4.4 DMI

`get_direct_mem_ptr` always returns `false` and `invalidate_direct_mem_ptr` is a no-op: register reads have side effects (RBR pop, IIR/MSR clear-on-read), so DMI must never be granted.

5. Register decode

5.1 DLAB-aware decode table

Decode is a switch on `addr` with a special case for offsets `0x00` and `0x04` gated by `LCR.DLAB`:

```
bool uart::reg_read(uint64_t off, uint32_t& data) {
    const bool dlab = (regs_.lcr >> 7) & 0x1;
    switch (off) {
        case 0x00: data = dlab ? regs_.dll : rbr_read(); break;
        case 0x04: data = dlab ? regs_.dlm : regs_.ier; break;
        case 0x08: data = iir_read(); break; // IIR (R0)
        case 0x0C: data = regs_.lcr; break;
        case 0x10: data = regs_.mcr; break;
        case 0x14: data = lsr_read(); break; // clears sticky errs
        case 0x18: data = msr_read(); break; // clears delta bits
        case 0x1C: data = regs_.scr; break;
        case 0x20: data = regs_.ecr; break;
        case 0x24: data = regs_.itr; break;
        default: return false; // decode miss
    }
    return true;
}
```

`reg_write` mirrors this, with `0x00/0x04` writing DLL/DLM (DLAB=1) or THR/IER (DLAB=0), and `0x08` writing FCR (which shares the address with the read-only IIR).

5.2 Read side effects

Register	Read side effect
RBR (0x00, DLAB=0)	Pops one character from the RX FIFO; updates <code>LSR.DR</code> ; may clear data-ready interrupt
IIR (0x08)	If the pending source is data-ready, clears it
LSR (0x14)	Clears the sticky OE/PE/FE/BI bits after the value is captured
MSR (0x18)	Clears the delta bits DCTS/DDSR/TERI/DDCD and latches the current modem levels

5.3 Write side effects

Register	Write side effect
THR (0x00, DLAB=0)	Pushes a character into the TX FIFO; clears <code>LSR.THRE</code> ; schedules a recompute so the character is “transmitted”
FCR (0x08)	FIFO enable/disable resets both FIFOs; explicit RX/TX reset bits are self-clearing
LCR (0x0C)	Updates word format and DLAB; toggling DLAB changes the decode of 0x00/0x04
IER (0x04)	Recompute interrupt outputs
MCR (0x10)	Updates modem outputs / loopback mode; recompute
ITR (0x24)	Forces interrupt sources for test; recompute

6. Internal data structures

6.1 Register file

```

struct uart_regs {
    uint8_t ier = 0;    // Interrupt Enable
    uint8_t fcr = 0;    // FIFO Control (write-only shadow)
    uint8_t lcr = 0;    // Line Control
    uint8_t mcr = 0;    // Modem Control
    uint8_t lsr = 0x60; // Line Status (THRE=1, TEMT=1 at reset)
    uint8_t msr = 0;    // Modem Status
    uint8_t scr = 0;    // Scratch
    uint8_t ecr = 0;    // Extended Control
    uint8_t itr = 0;    // Interrupt Test
    uint8_t dll = 0;    // Divisor Latch LSB
    uint8_t dlm = 0;    // Divisor Latch MSB
};

```

6.2 FIFO entries

Each RX FIFO entry carries the character plus its per-character error flags, mirroring the hardware behaviour where errors travel with the character to the top of the FIFO:

```

struct rx_entry {
    uint8_t character = 0;
    bool parity_err = false;
    bool framing_err = false;
    bool break_err = false;
};

std::deque<rx_entry> rx_fifo; // sized/limited by rx_fifo_depth_p_
std::deque<uint8_t> tx_fifo; // sized/limited by tx_fifo_depth_p_

```

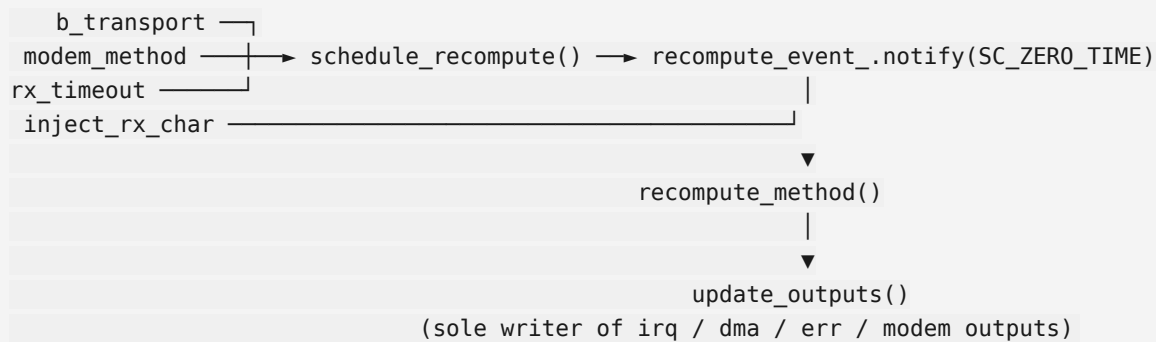
When FIFO mode is disabled (`FCR.FIFO_ENABLE = 0`) the dequeues are limited to a single entry, modelling the single-byte RBR / THR.

6.3 Derived/runtime state

```
bool    fifo_en_      = false; // current FCR.FIFO_ENABLE
unsigned rx_trigger_   = 1;     // decoded 4-bit trigger level (chars)
uint8_t dma_mode_     = 0;     // 0 or 1
// Modem "last read" snapshots for delta detection
bool    cts_last_=false, dsr_last_=false, dcd_last_=false, ri_prev_=false;
// DMA Mode-1 FSM state
enum class dma1_rx_state { IDLE, READY } dma1_rx_ = dma1_rx_state::IDLE;
enum class dma1_tx_state { IDLE, NOT_READY } dma1_tx_ = dma1_tx_state::IDLE;
```

7. Process and event topology

The model follows the SMC **single-driver discipline**: every `sc_out` port is written by exactly one function (update_outputs). All state-changing paths (`b_transport`, the modem method, the timeout method) converge on `schedule_recompute()`, which notifies `recompute_event_`; `recompute_method` then calls `update_outputs` exactly once per delta.



This avoids multiple-driver conflicts that SystemC 3.0 flags as errors and keeps the output evaluation order deterministic.

7.1 RX timeout timer

When a character is pushed to the RX FIFO and the FIFO transitions to non-empty, `rx_timeout_event_.notify(timeout_delay)` is (re)armed. Each new character cancels and re-arms it. If it expires with the FIFO still non-empty, the timeout interrupt is raised. The `timeout_delay` is a coarse functional approximation (configurable; default derived from `access_delay_ns`), not the exact 4-character-time of real hardware.

8. Behavioural algorithms

8.1 Transmit

```

on THR write:
    if tx_fifo.size() < depth:           // drop silently if full
        tx_fifo.push_back(byte)
    regs_.lsr &= ~THRE
    schedule_recompute()

in recompute (functional "transmit"):
    while !tx_fifo.empty():
        byte = tx_fifo.pop_front()
        frame = apply_framing(byte, lcr) // parity / stop bits computed
        tx_history_.push_back(byte)      // observable via dbg_tx_pop()
        if mcr.LOOP (system loopback):
            deliver byte back to RX path
    regs_.lsr |= THRE | TEMT             // FIFO + serialiser empty
    if ier.ETBEI: raise THRE interrupt

```

Because the model is loosely-timed, the entire TX FIFO drains within one recompute; the THRE/TEMT bits therefore settle to “empty” immediately after the transaction, which is consistent with software that polls THRE before each write.

8.2 Receive

```

on inject_rx_char(ch, perr, ferr, berr) (or system-loopback delivery):
    if rx_fifo.size() == depth:
        regs_.lsr |= OE                 // overrun, drop new char
    else:
        rx_fifo.push_back({ch, perr, ferr, berr})
    regs_.lsr |= DR
    rearm_rx_timeout()
    schedule_recompute()

on RBR read:
    if !rx_fifo.empty():
        e = rx_fifo.front(); rx_fifo.pop_front()
        update LSR.PE/FE/BI from new front entry // errors track top-of-FIFO
        if rx_fifo.empty(): regs_.lsr &= ~DR
        return e.character
    return 0x00

```

8.3 Interrupt arbitration

```
uint8_t uart::interrupt_id() const {
    // ITR test bits FORCE the corresponding source active (bypassing the real
    // condition and the IER enable); otherwise a source is active only when
    // its condition holds AND its IER enable is set.
    if (itr_TFEI() || (fifo_error() && ier_EFEI())) return 0x7;
    if (itr_TLSI() || (line_status() && ier_ELSI())) return 0x3;
    if (itr_TRTI() || (rx_timeout() && ier_ERBFI())) return 0x6;
    if (itr_TRBFI() || (rx_data_ready() && ier_ERBFI())) return 0x2;
    if (itr_TTBEI() || (thre() && ier_ETBEI())) return 0x1;
    if (itr_TDSSI() || (modem_status() && ier_EDSSI())) return 0x0;
    return 0xff; // none
}
```

`update_outputs` drives `irq_o = (interrupt_id() != 0xff)` and writes `IIR.INTERRUPT_PENDING = ~irq`, `IIR.INTERRUPT_ID = id`, `IIR.FIFOS_ENABLED = fifo_en?3:0`.

8.4 DMA handshake

```
Mode 0:
    rxrdy_o = !rx_fifo.empty()
    txrdy_o = tx_fifo.size() < depth

Mode 1 (fifo_en required):
    RX FSM: IDLE --(watermark||timeout)--> READY;  READY --(fifo empty)--> IDLE
            rxrdy_o = (state == READY)
    TX FSM: IDLE --(fifo full)--> NOT_READY;      NOT_READY --(fifo empty)--> IDLE
            txrdy_o = (state == IDLE)
```

8.5 Modem control / loopback

`update_outputs` computes the modem outputs from MCR and the loopback mode, and the MSR input bits from the modem inputs and the loopback mode, exactly as tabulated in `01_UART_Specification.md` §8.4. `modem_method` additionally latches delta bits (DCTS/DDSR/DDCD) and the trailing-edge RI (TERI) by comparing against the snapshot taken at the last MSR read.

9. Reset implementation

`reset_proc` is an `SC_METHOD` sensitive to `rst_n_i`. On the falling edge (active-low assert) it:

```

void uart::reset_proc() {
    if (rst_n_i.read() == false) { // reset asserted
        regs_ = uart_regs{}; // LSR back to 0x60, all else 0
        tx_fifo_.clear();
        rx_fifo_.clear();
        tx_history_.clear();
        fifo_en_ = false;
        dma1_rx_ = dma1_rx_state::IDLE;
        dma1_tx_ = dma1_tx_state::IDLE;
        rx_timeout_event_.cancel();
        schedule_recompute(); // drive outputs to reset values
    }
}

```

Reset is asynchronous (level-sensitive on `rst_n_i`). On de-assert the model simply resumes from the cleared state. `update_outputs` then drives `tx_o = 1`, all modem outputs to 1 (deasserted), and `irq/dma/err` to 0.

10. Loosely-timed timing model and quantum keeper

10.1 Timing philosophy

The UART is a **loosely-timed** TLM-2.0 target. It owns no clock and models no bit-level serial timing. Time enters the model in exactly one place: the `delay` argument of `b_transport`, to which the model adds `access_delay_ns` (default 2 ns) to approximate AXI4-Lite register-access latency.

10.2 Role of the quantum keeper

The UART target itself does **not** instantiate a quantum keeper — temporal decoupling is the initiator's responsibility. The model is written to be correct under an LT initiator that batches time using a `tlm_utils::tlm_quantumkeeper`:

- The model **only reads** the incoming `delay` and **adds** its own access latency; it never calls `wait()` inside `b_transport`. This keeps it compatible with a decoupled initiator that runs ahead of simulation time by up to one global quantum.
- All asynchronous, time-consuming effects (RX timeout) are modelled with `sc_event::notify(delay)` rather than blocking waits, so they are scheduled on the SystemC kernel timeline independently of the initiator's quantum.
- Because the TX FIFO drains within a single recompute (zero simulated serial time), there is no risk of the initiator's quantum "hiding" a transmit; software polling THRE/TEMT always observes a consistent state immediately after its write completes.

10.3 Test-bench initiator pattern

The test bench drives transactions through a quantum keeper to exercise the LT path:

```

    tlm_utils::tlm_quantumkeeper qk;
    qk.set_global_quantum(sc_time(1, SC_US));
    qk.reset();
    // ... per transaction:
    sc_time delay = qk.get_local_time();
    socket->b_transport(gp, delay);    // model adds access_delay_ns to delay
    qk.set_and_sync(delay);           // sync if quantum exceeded

```

This confirms the model behaves correctly whether the initiator syncs every transaction or batches many transactions within one quantum.

11. Error handling implementation

- **Bus errors** are mapped to TLM response codes as in §4.2; the model never throws on a bad transaction.
- **RX character errors** (parity/framing/break) are attached to the FIFO entry at injection time and surface through LSR as each character reaches the top of the FIFO.
- **Overrun** is detected at injection: a push to a full RX FIFO sets `LSR.OE` and drops the new character (FIFO mode) or overwrites the held byte (non-FIFO mode).
- `err_o` is the OR of the active unmasked RX error conditions, driven by `update_outputs`.

12. Debug and verification hooks

Hook	Use
<code>inject_rx_char()</code>	Primary RX stimulus path for the test bench (no serial line needed).
<code>dbg_tx_pop()</code> / <code>dbg_tx_count()</code>	Capture and check transmitted characters.
<code>dbg_rx_count()</code>	Assert FIFO occupancy.
<code>dbg_reg()</code>	Side-effect-free register checks (does not pop RBR or clear sticky bits).
<code>transport_dbg</code>	Same as <code>dbg_reg</code> but through the socket; used by integrated debuggers.
<code>dump_state()</code>	One-shot human-readable dump (registers, FIFO depths, FSM states).

13. Integration with `smc_top`

1. Instantiate one `smc::uart` per console/debug port (default 4 in the SMC platform).
2. Bind `reg_socket` into the fabric decoder at the UART's base address with a 256-byte window.
3. Connect `rst_n_i` to the platform reset, `irq_o` to the PLIC source input assigned to that UART, and the modem/DMA outputs as required (tie off unused inputs to their deasserted level: `cts_ni=dsr_ni=ri_ni=dcd_ni=1`).
4. Optionally set CCI presets for `tx_fifo_depth` / `rx_fifo_depth` before elaboration.

```

smc::uart uart0("uart0");
fabric.bind(uart0.reg_socket, UART0_BASE, smc::uart_cfg::WINDOW_SIZE);
uart0.rst_n_i(rst_n);
uart0.irq_o(plic_src[UART0_IRQ_ID]);
// tie off modem inputs if unused
uart0.cts_ni(const_high); uart0.dsr_ni(const_high);
uart0.ri_ni(const_high); uart0.dcd_ni(const_high);

```

14. Modeling decisions and trade-offs

Decision	Rationale
Character-granularity TX/RX instead of bit-level serialisation	The functional model targets firmware bring-up and console I/O; bit timing adds cost without firmware-visible value.
RX stimulus via <code>inject_rx_char()</code> back door	Avoids modelling a serial line and a second UART; lets the test bench precisely control timing and error flags.
Single <code>update_outputs</code> writer	Satisfies SystemC 3.0 single-driver rule and makes output ordering deterministic.
TX FIFO drains in one recompute	Matches LT abstraction; software always sees consistent THRE/TEMT after its write.
No quantum keeper inside the target	Temporal decoupling belongs to the initiator; target only annotates <code>delay</code> .
RX timeout as coarse <code>sc_event</code>	Provides the timeout <i>behaviour</i> (interrupt + IIR=0x6) without bit-accurate character timing.

15. Known limitations and future work

- **No bit-level timing.** Baud rate affects nothing except the divisor-zero disable; two UART models cannot be wired serially and expected to handshake at a real baud rate.
- **RX timeout is approximate.** The interrupt fires, but its latency is a functional approximation, not 4 character-times.
- **No noise/glitch modelling.** The majority filter and input synchronisers are omitted.
- **Future:** an optional approximately-timed (AT) profile could add per-character serial delays for performance studies; the register and interrupt logic would be unchanged.